Databases – Exercises SQL

Vrije Universiteit Amsterdam

1. Consider the following relational database:

   Person(<u>id</u>, name, city, age)

   Knows(<u>id1</u> $\rightarrow$ Person, <u>id2</u> $\rightarrow$ Person)

   Likes(<u>id1</u> $\rightarrow$ Person, <u>id2</u> $\rightarrow$ Person)

   Friend(<u>id1</u> $\rightarrow$ Person, <u>id2</u> $\rightarrow$ Person)

   Account(<u>accountNr</u>, balance)

   BelongsTo(<u>id</u> $\rightarrow$ Person, <u>accountNr</u> $\rightarrow$ Account)

   We assume that the *friends* relation is **symmetric**. If (A, B) is a tuple in the Friend table, then A is friend of B and B is friend of A. We do *not* assume that the relations *knows* and *likes* are symmetric. If (A, B) is a tuple in the Knows table, then A knows B, but B does not necessarily know A.

   Give an expression in SQL to express each of the following queries:

   (a) Find the name of all persons that are younger than 40.

   Solution:

   ```
   select name
   from   Person
   where  age < 40
   ```

   (b) Find the name of all persons that have a friend.

   Solution:

   ```
   select name
   from   Person P, Friend F
   where  (P.id = F.id1) or (P.id = F.id2)
   ```

   i. Will the query return duplicate results?

   Solution: Yes, someone who has multiple friends will be returned multiple times.

   ii. Does adding distinct solve the problem?

   Solution: No. What distinct does is eliminate all duplicates from the final result. In the case there are two people with friends, that have the same name, only one name will be returned. This is not what we intend.

iii. Improve the query by using `in`.

Solution:

```
select name
from   Person P
where  (P.id in (select id1 from Friend))
    or (P.id in (select id2 from Friend))
```

iv. Do the same with `exists`.

Solution:

```
select name
from   Person P
where  exists
         (select *
          from   Friend F
          where  P.id = F.id1 or P.id = F.id2)
```

(c) Find the name of all people that have no friends using `not in`.

Solution:

```
select name
from   Person P
where  (P.id not in (select id1 from Friend))
   and (P.id not in (select id2 from Friend))
```

Note the very similar structure with the query that asks to find all people that have at least one friend. No friends, means there is not even one friend. So, we get the result by just negating the `in` clause. Same reasoning with the `exists` version.

(d) Find the name of all people that have no friends using `not exits`.

Solution:

```
select name
from   Person P
where  not exists
         (select *
          from   Friend F
          where  P.id = F.id1 or P.id = F.id2)
```

(e) Find the name of all persons that have a bank account that contains more than 1000 euros.

Solution:

```
select name
from   Person P, Account A, BelongsTo B
where  P.id = B.id and B.accountNr = A.accountNr
```

```
   and A.balance > 1000
```

i. Does the above query return duplicates?

Solution: Yes.

ii. Does adding distinct solve the problem?

Solution: No. It eliminates also distinct people with the same name.

iii. Do the query using `in`.

Solution:

```
select name
from   Person P
where  P.id in
         (select B.id
          from   BelongsTo B, Account A
          where  B.accountNr = A.accountNr
             and A.balance > 1000)
```

(f) Find all people that know at least two people. Do not use `group by`.

Solution:

```
select name
from   Person P, Knows K1, Knows K2
where  P.id = K1.id1
   and P.id = K2.id1
   and K1.id2 <> K2.id2
```

(g) Find all people that know at least two people using `group by`.

Solution:

```
select   name
from     People P, Knows K
where    P.id = K.id1
group by P.id, name
having count(*) >= 2
```

2. Consider the following relational database:

   Employee(employeeName, street, city)

   Works(employeeName → Employee, companyName → Company, salary)

   Company(companyName, city)

   Manages(employeeName → Employee, managerName → Employee)

   Give an expression in SQL to express each of the following queries:

   (a) Find the names and the cities of residence of all employees who work for the "First Bank Corporation".

   Solution:

```
select employeeName, city
from   Employee E, Works W
where  E.employeeName = W.employeeName
   and W.companyName = 'First Bank Corporation'
```

   (b) Find the names, street addresses, and cities of residence of all employees who work for "First Bank Corporation" and earn more than $10,000.

   Solution:

```
select employeeName, street, city
from   Employee E, Works W
where  E.employeeName = W.employeeName
   and W.companyName = 'First Bank Corporation'
   and salary > 1000
```

   (c) Find all employees in the database who do not work for "First Bank Corporation".

   Solution:

```
select employeeName
from   Works W
where  W.companyName <> 'First Bank Corporation'
```

   (d) Find all employees in the database who earn more than each employee of "Small Bank Corporation".

   Solution:

```
select employeeName
from   Works W
where  salary > all
        (select salary
         from   Works
         where  companyName = 'Small Bank Corporation')
```

(e) Assume that the companies may be located in several cities. Find all companies located only in the cities in which the "Small Bank Corporation" (SBC) is located.
For example, assume SBC is located in Amsterdam and Eindhoven. Then we want all the companies that are located in any of these two cities or in both.

Solution: It helps to first look in which cities the SBC is located. Then we can easily get the names of the companies which are located in a city that is not one of the SBC's cities. Finally, we only retrieve the companies that do not make part of this list. This gives rise to two times nested `not in` or `not exists`.
Observe that we cannot use a straightforward check for the names of companies that are located in the cities of SBC. Because, for example, a company located in Amsterdam and Utrecht, would be selected.

```
select companyName
from   Company
where  companyName not in
         (select companyName
          from   Company
          where  city not in
            (select city
             from   Company
             where  companyName = 'SBC')
```

Assume these are the list of companies and the cities in which they are related.

companyMame	city
SBC	Amsterdam
SBC	Eindhoven
MBC	Amsterdam
MBC	Utrecht
INC	Amsterdam
ABT	Eidhoven
ABT	Amsterdam

The innermost `select` statement will give:

city
Amsterdam
Eindhoven

The middle select query will then show:

companyName
MBC

Finally the outer SELECT statement will show:

company
SBC
SBC
INC
ABT
ABT